

Skyflow for Snowflake - Deployment Guide

Complete step-by-step guide to deploy Skyflow tokenization and detokenization for Snowflake using AWS Lambda.

Table of Contents

- [Prerequisites](#)
 - [Required Tools](#)
 - [Required Information](#)
 - [Deployment Steps](#)
 - [Step 1: Configure AWS CLI](#)
 - [Step 2: Prepare Configuration Files](#)
 - [Configuration Method Selection](#)
 - [Step 3: Create AWS Secrets Manager Secret](#)
 - [Step 4: Create Lambda IAM Role](#)
 - [Step 5: Build Lambda Deployment Package](#)
 - [Step 6: Create Lambda Function](#)
 - [Step 7: Create API Gateway](#)
 - [Step 8: Create Snowflake IAM Role](#)
 - [Step 9: Configure Snowflake](#)
 - [Testing](#)
 - [Test Lambda Directly](#)
 - [Test in Snowflake](#)
 - [Check CloudWatch Logs](#)
 - [Performance Features](#)
 - [Configuration Options](#)
 - [Troubleshooting](#)
 - [Lambda Can't Read Secret](#)
 - [Permission Denied](#)
 - [HTTP 401 Unauthorized](#)
 - [Snowflake Functions Not Working](#)
 - [High Latency](#)
 - [Updating the Lambda](#)
 - [Cleanup / Uninstall](#)
 - [Support](#)
 - [Appendix: Manual Environment Variables](#)
 - [Additional Resources](#)
-

Prerequisites

Required Tools

- **Node.js 18+** - [Download](#)

- **AWS CLI** - [Install Guide](#)
 - **Note:** This guide uses AWS CLI for all deployment steps. If you prefer Infrastructure as Code (IaC), you can adapt these steps to Terraform, CloudFormation, or AWS CDK.
- **AWS Account** with permissions for Lambda, API Gateway, IAM, Secrets Manager
- **Snowflake Account** with ACCOUNTADMIN role
- **Skyflow Account** with vault access and authentication credentials (JWT or API Key)

Required Information

Before starting, gather these values:

From Skyflow:

- Vault URL (e.g., <https://your-cluster-id.vault.skyflowapis.com>)
- **Authentication credentials** (choose one):
 - **JWT (Service Account):** Client ID, private key, token URI, key ID (recommended)
 - **API Key:** Bearer token
- Vault IDs for each data type (NAME, ID, DOB, SSN)
- Table and column names for each data type

From AWS:

- AWS Account ID
- AWS Region (recommend: [us-east-1](#))

From Snowflake:

- Account identifier (e.g., [ABC12345.us-east-1](#))
- Username with ACCOUNTADMIN role
- Password
- Database and schema names
- Warehouse name



Deployment Steps

Step 1: Configure AWS CLI and Region

```
# Configure AWS credentials
aws configure

# Enter:
# - AWS Access Key ID
# - AWS Secret Access Key
# - Default region (recommend: us-east-1, but any region works)
# - Default output format (json)

# Verify configuration
aws sts get-caller-identity
```

Set deployment region (used throughout this guide):

```
# Set your desired AWS region (default: us-east-1)
export AWS_REGION=${AWS_REGION:-us-east-1}

echo "Deploying to region: $AWS_REGION"
```

Region Considerations:

- **us-east-1** (default)
- **eu-west-1**: Better for European data residency requirements
- **ap-southeast-1**: Better for Asia-Pacific deployments
- All AWS regions support Lambda, API Gateway, and Secrets Manager

Note: Make sure your Snowflake deployment can reach your chosen AWS region's API Gateway endpoints.

Step 2: Prepare Configuration Files

2.1 Create configuration file

Create `skyflow-config.json` with your Skyflow credentials. You can use either **JWT (Service Account)** or **API Key** authentication. The system will auto-detect which credentials are present (JWT is checked first).

Option A: JWT (Service Account) Authentication (recommended)

```
{
  "credentials": {
    "clientID": "YOUR_CLIENT_ID",
    "clientName": "YOUR_SERVICE_ACCOUNT_NAME",
    "tokenURI": "https://manage.skyflowapis.com/v1/auth/sa/oauth/token",
    "keyID": "YOUR_KEY_ID",
    "privateKey": "-----BEGIN PRIVATE KEY-----\nYOUR_PRIVATE_KEY_HERE\n---
--END PRIVATE KEY-----\n",
    "keyAlgorithm": "KEY_ALG_RSA_2048"
  },
  "vaults": {
    "vaultUrl": "https://YOUR_CLUSTER_ID.vault.skyflowapis.com",
    "definitions": [
      {
        "vaultId": "YOUR_VAULT_ID_FOR_NAMES",
        "table": "persons",
        "column": "name",
        "dataType": "NAME",
        "transformations": {
          "uppercase": true,
          "minLength": 3,
          "stripPunctuation": true
        }
      }
    ]
  }
}
```

```

    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_IDS",
      "table": "persons",
      "column": "person_id",
      "dataType": "ID",
      "transformations": {
        "uppercase": true,
        "minLength": 3,
        "stripPunctuation": true
      }
    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_DOBS",
      "table": "persons",
      "column": "date_of_birth",
      "dataType": "DOB",
      "transformations": {
        "uppercase": false,
        "stripPunctuation": true,
        "validation": {
          "minDate": "0600-01-01",
          "maxDate": "3337-11-27"
        }
      }
    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_SSNS",
      "table": "persons",
      "column": "ssn",
      "dataType": "SSN",
      "transformations": {
        "uppercase": false,
        "minLength": 3,
        "stripPunctuation": true
      }
    }
  ]
},
"tokenizeBatchSize": 5,
"tokenizeMaxConcurrency": 400,
"detokenizeBatchSize": 300,
"detokenizeMaxConcurrency": 400,
"logLevel": "INFO"
}

```

Option B: API Key Authentication

```

{
  "credentials": {
    "apiKey": "YOUR_SKYFLOW_API_KEY_HERE"
  },

```

```
"vaults": {
  "vaultUrl": "https://YOUR_CLUSTER_ID.vault.skyflowapis.com",
  "definitions": [
    {
      "vaultId": "YOUR_VAULT_ID_FOR_NAMES",
      "table": "persons",
      "column": "name",
      "dataType": "NAME",
      "transformations": {
        "uppercase": true,
        "minLength": 3,
        "stripPunctuation": true
      }
    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_IDS",
      "table": "persons",
      "column": "person_id",
      "dataType": "ID",
      "transformations": {
        "uppercase": true,
        "minLength": 3,
        "stripPunctuation": true
      }
    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_DOBS",
      "table": "persons",
      "column": "date_of_birth",
      "dataType": "DOB",
      "transformations": {
        "uppercase": false,
        "stripPunctuation": true,
        "validation": {
          "minDate": "0600-01-01",
          "maxDate": "3337-11-27"
        }
      }
    },
    {
      "vaultId": "YOUR_VAULT_ID_FOR_SSNS",
      "table": "persons",
      "column": "ssn",
      "dataType": "SSN",
      "transformations": {
        "uppercase": false,
        "minLength": 3,
        "stripPunctuation": true
      }
    }
  ]
},
"tokenizeBatchSize": 5,
"tokenizeMaxConcurrency": 400,
```

```
"detokenizeBatchSize": 300,  
"detokenizeMaxConcurrency": 400,  
"logLevel": "INFO"  
}
```

Configuration Notes:

- Replace all `YOUR_*` placeholder values with your actual Skyflow configuration
 - The `vaultUrl` should include your cluster ID (e.g., `https://abc123.vault.skyflowapis.com`)
 - Batch sizes and concurrency can be tuned based on your workload (see [Configuration Options](#))
-

Configuration Method Selection

There are **two configuration methods** available, both fully documented in this guide:

Method 1: AWS Secrets Manager Recommended

Store configuration in AWS Secrets Manager and reference it via environment variable.

Pros:

- Configuration updates without Lambda redeployment
- Centralized secrets management with audit logging
- Automatic encryption at rest
- Supports credential rotation with 5-minute cache refresh

Cons:

- Additional AWS service dependency
- Minimal cost (~\$0.40/month per secret)
- Requires IAM permissions for Secrets Manager

Use when: You need configuration flexibility, credential rotation, or are deploying to production.

Setup: Follow Step 3 (Create Secret) → Step 6 (set `SECRETS_MANAGER_SECRET_NAME` env var)

Method 2: Lambda Environment Variables

Store configuration directly as Lambda environment variables (no Secrets Manager).

Pros:

- Simpler setup (no Secrets Manager)
- No additional AWS costs
- Faster cold starts (no Secrets Manager API call)
- All AWS CLI commands, no external dependencies

Cons:

- Configuration changes require Lambda redeployment

- More complex initial setup (many environment variables)
- 4KB limit on total environment variable size
- Less secure (visible in Lambda console)

Use when: You're doing development/testing, have simple configuration, or want to avoid Secrets Manager.

Setup: Skip Step 3 → Step 6 (set multiple `SKYFLOW_*` env vars instead) → See "Appendix: Manual Environment Variables" below

This guide follows Method 1 (Secrets Manager). If you want Method 2, see the "Appendix: Manual Environment Variables" section at the end of this guide.

Step 3: Create AWS Secrets Manager Secret

```
# Set your secret name (customize this if needed)
export SECRET_NAME="skyflow-tokenization-config"

# Create the secret in your chosen region
aws secretsmanager create-secret \
  --name ${SECRET_NAME} \
  --description "Skyflow tokenization configuration" \
  --secret-string file://skyflow-config.json \
  --region ${AWS_REGION}

# Verify secret was created
aws secretsmanager describe-secret \
  --secret-id ${SECRET_NAME} \
  --region ${AWS_REGION}
```

Note: The secret name is configurable. The Lambda reads it from the `SECRETS_MANAGER_SECRET_NAME` environment variable (Step 6).

Expected output:

```
{
  "ARN":
  "arn:aws:secretsmanager:${AWS_REGION}:YOUR_ACCOUNT:secret:skyflow-
  tokenization-config-XXXXXX",
  "Name": "skyflow-tokenization-config",
  "Description": "Skyflow tokenization configuration",
  ...
}
```

Step 4: Create Lambda IAM Role

4.1 Create trust policy

Create `lambda-trust-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

4.2 Create the role

```
# Get your AWS account ID (AWS_REGION already set in Step 1)
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --
output text)

# Create IAM role
aws iam create-role \
  --role-name skyflow-tokenization-lambda-role \
  --assume-role-policy-document file://lambda-trust-policy.json \
  --description "Execution role for Skyflow tokenization Lambda"

# Attach basic Lambda execution policy
aws iam attach-role-policy \
  --role-name skyflow-tokenization-lambda-role \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole
```

4.3 Add Secrets Manager permissions

Create `secrets-manager-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": "arn:aws:secretsmanager:*:*:secret:skyflow-tokenization-
config-*"
    }
  ]
}
```



```
]
}
```

Note: The `-*` suffix in the Resource ARN matches AWS's automatic random string. If you used a custom secret name, update the policy to match (e.g., `secret:my-custom-name-*`). Using `*` for region allows the Lambda to work if you move it to another region later.

```
# Attach Secrets Manager policy
aws iam put-role-policy \
    --role-name skyflow-tokenization-lambda-role \
    --policy-name SecretsManagerReadPolicy \
    --policy-document file://secrets-manager-policy.json

# Wait for role to propagate
sleep 10
```

Step 5: Build Lambda Deployment Package

5.1 Install dependencies

```
# Make sure you're in the directory with the Lambda code
# Directory should contain: config.js, skyflow-client.js, handler.js,
package.json

npm install --production
```

5.2 Create deployment ZIP

```
# Create deployment package
zip -r function.zip config.js skyflow-client.js handler.js package.json
node_modules/

# Verify package size (should be ~5-10 MB)
ls -lh function.zip
```

Step 6: Create Lambda Function

```
# Create Lambda function (uses SECRET_NAME from Step 3)
aws lambda create-function \
    --function-name skyflow-tokenization \
    --runtime nodejs20.x \
    --role arn:aws:iam::${AWS_ACCOUNT_ID}:role/skyflow-tokenization-
```

```

lambda-role \
  --handler handler.handler \
  --zip-file fileb://function.zip \
  --timeout 60 \
  --memory-size 256 \
  --description "Skyflow tokenization and detokenization for Snowflake"
\
  --environment Variables="{SECRETS_MANAGER_SECRET_NAME=${SECRET_NAME}}"
\
  --region ${AWS_REGION}

# Verify Lambda was created
aws lambda get-function \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --query 'Configuration.[FunctionName,Runtime,Handler,State]'

```

Note: The `SECRETS_MANAGER_SECRET_NAME` must match the secret name from Step 3.

Expected output:

```

[
  "skyflow-tokenization",
  "nodejs20.x",
  "handler.handler",
  "Active"
]

```

Step 7: Create API Gateway

This implementation uses a **single API Gateway endpoint** (`/process`) that receives operation and data type via HTTP headers, with caller context automatically provided by Snowflake's `CONTEXT_HEADERS`.

7.1 Create REST API

```

# Create API Gateway
API_ID=$(aws apigateway create-rest-api \
  --name skyflow-tokenization-api \
  --description "API for Skyflow tokenization and detokenization" \
  --endpoint-configuration types=REGIONAL \
  --region ${AWS_REGION} \
  --query 'id' \
  --output text)

echo "API ID: ${API_ID}"

# Get root resource ID
ROOT_RESOURCE_ID=$(aws apigateway get-resources \

```

```

--rest-api-id ${API_ID} \
--region ${AWS_REGION} \
--query 'items[?path==`/`].id' \
--output text)

echo "Root Resource ID: ${ROOT_RESOURCE_ID}"

```

7.2 Create single /process resource

```

# Create /process resource (single endpoint for all operations)
PROCESS_RESOURCE_ID=$(aws apigateway create-resource \
  --rest-api-id ${API_ID} \
  --parent-id ${ROOT_RESOURCE_ID} \
  --path-part process \
  --region ${AWS_REGION} \
  --query 'id' \
  --output text)

echo "Process Resource ID: ${PROCESS_RESOURCE_ID}"

# Create POST method
aws apigateway put-method \
  --rest-api-id ${API_ID} \
  --resource-id ${PROCESS_RESOURCE_ID} \
  --http-method POST \
  --authorization-type NONE \
  --region ${AWS_REGION}

# Set up Lambda integration (AWS_PROXY automatically passes headers)
aws apigateway put-integration \
  --rest-api-id ${API_ID} \
  --resource-id ${PROCESS_RESOURCE_ID} \
  --http-method POST \
  --type AWS_PROXY \
  --integration-http-method POST \
  --uri "arn:aws:apigateway:${AWS_REGION}:lambda:path/2015-03-
31/functions/arn:aws:lambda:${AWS_REGION}:${AWS_ACCOUNT_ID}:function:skyfl
ow-tokenization/invocations" \
  --region ${AWS_REGION}

echo "Configured single endpoint: /process"

```

7.3 Grant API Gateway permission to invoke Lambda

```

# Add Lambda permission for API Gateway
aws lambda add-permission \
  --function-name skyflow-tokenization \
  --statement-id apigateway-invoke \
  --action lambda:InvokeFunction \

```

```
--principal apigateway.amazonaws.com \
--source-arn "arn:aws:execute-
api:${AWS_REGION}:${AWS_ACCOUNT_ID}:${API_ID}/*" \
--region ${AWS_REGION}
```

7.4 Deploy API

```
# Deploy API to prod stage
aws apigateway create-deployment \
  --rest-api-id ${API_ID} \
  --stage-name prod \
  --description "Production deployment" \
  --region ${AWS_REGION}

# Get API URL
API_URL="https://${API_ID}.execute-
api.${AWS_REGION}.amazonaws.com/prod/process"
echo "API URL: ${API_URL}"
echo ""
echo "Note: Operation and data type are passed via HTTP headers (X-
Operation, X-Data-Type)"
echo "      Snowflake functions automatically include these headers in
requests"
```

Step 8: Create Snowflake IAM Role

8.1 Create trust policy (placeholder)

Create `snowflake-trust-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::YOUR_ACCOUNT_ID:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "PLACEHOLDER_WILL_BE_UPDATED"
        }
      }
    }
  ]
}
```

Replace `YOUR_ACCOUNT_ID` with your AWS account ID, then:

```
# Create Snowflake IAM role
aws iam create-role \
  --role-name SnowflakeAPIRole \
  --assume-role-policy-document file://snowflake-trust-policy.json \
  --description "IAM role for Snowflake API integration"
```

8.2 Attach API Gateway invoke policy

Create `api-invoke-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-
api:YOUR_REGION:YOUR_ACCOUNT_ID:YOUR_API_ID/*"
    }
  ]
}
```

Replace `YOUR_REGION`, `YOUR_ACCOUNT_ID`, and `YOUR_API_ID` with your values, then:

```
# Attach policy to role
aws iam put-role-policy \
  --role-name SnowflakeAPIRole \
  --policy-name APIGatewayInvokePolicy \
  --policy-document file://api-invoke-policy.json

# Get role ARN (save this for Snowflake)
SNOWFLAKE_ROLE_ARN=$(aws iam get-role \
  --role-name SnowflakeAPIRole \
  --query 'Role.Arn' \
  --output text)

echo "Snowflake Role ARN: ${SNOWFLAKE_ROLE_ARN}"
```

Step 9: Configure Snowflake

9.1 Create API Integration

Connect to Snowflake and run:

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE API INTEGRATION skyflow_api_integration
  API_PROVIDER = aws_api_gateway
  API_AWS_ROLE_ARN =
'arn:aws:iam::YOUR_ACCOUNT_ID:role/SnowflakeAPIRole'
  ENABLED = TRUE
  API_ALLOWED_PREFIXES = ('https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/');

-- Get trust policy values
DESC API INTEGRATION skyflow_api_integration;
```

9.2 Update AWS IAM Trust Policy

From the Snowflake output, copy:

- `API_AWS_IAM_USER_ARN`
- `API_AWS_EXTERNAL_ID`

Update `snowflake-trust-policy.json`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/abc123-s"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "ABC12345_SFCTestRole=1_xyz789"
        }
      }
    }
  ]
}
```

Update the trust policy in AWS:

```
aws iam update-assume-role-policy \
  --role-name SnowflakeAPIRole \
  --policy-document file://snowflake-trust-policy.json
```

9.3 Create External Functions

In Snowflake, run:

```
USE ROLE ACCOUNTADMIN;
USE DATABASE YOUR_DATABASE;
USE SCHEMA YOUR_SCHEMA;

-- All functions use a single /process endpoint with HTTP headers

-- Tokenization functions (with HEADERS and CONTEXT_HEADERS)
CREATE OR REPLACE EXTERNAL FUNCTION TOK_NAME(plaintext VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'tokenize', 'X-Data-Type' = 'NAME')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION TOK_ID(plaintext VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'tokenize', 'X-Data-Type' = 'ID')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION TOK_DOB(plaintext VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'tokenize', 'X-Data-Type' = 'DOB')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION TOK_SSN(plaintext VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'tokenize', 'X-Data-Type' = 'SSN')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

-- Detokenization functions (with HEADERS and CONTEXT_HEADERS)
CREATE OR REPLACE EXTERNAL FUNCTION DETOK_NAME(token VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'detokenize', 'X-Data-Type' = 'NAME')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
```

```

api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION DETOK_ID(token VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'detokenize', 'X-Data-Type' = 'ID')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION DETOK_DOB(token VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'detokenize', 'X-Data-Type' = 'DOB')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

CREATE OR REPLACE EXTERNAL FUNCTION DETOK_SSN(token VARCHAR)
  RETURNS VARCHAR
  API_INTEGRATION = skyflow_api_integration
  HEADERS = ('X-Operation' = 'detokenize', 'X-Data-Type' = 'SSN')
  CONTEXT_HEADERS = (CURRENT_USER, CURRENT_ROLE, CURRENT_ACCOUNT,
CURRENT_IP_ADDRESS)
  AS 'https://YOUR_API_ID.execute-
api.YOUR_REGION.amazonaws.com/prod/process';

-- Verify functions were created
SHOW FUNCTIONS LIKE 'TOK_%';
SHOW FUNCTIONS LIKE 'DETOK_%';

```

✓ Testing

Test Lambda Directly

```

# Test tokenization with headers
# Note: Snowflake prepends 'sf-custom-' to custom headers and 'sf-context-'
# to context headers
aws lambda invoke \
  --function-name skyflow-tokenization \
  --payload '{"headers":{"sf-custom-x-operation":"tokenize","sf-custom-
x-data-type":"NAME","sf-context-current-user":"TESTUSER","sf-context-
current-role":"TESTROLE","sf-context-current-account":"TESTACCT"},"body":
{"data":[{"id","John Doe"}]}' \
  --region ${AWS_REGION} \
  response.json

cat response.json

```



```
# Expected: {"statusCode":200,"body":{"\"data\":[[0,\"tok_...\"]]}}"}

# Test detokenization with headers (replace tok_abc123 with actual token
from above)
aws lambda invoke \
  --function-name skyflow-tokenization \
  --payload '{"headers":{"sf-custom-x-operation":"detokenize","sf-
custom-x-data-type":"NAME","sf-context-current-user":"TESTUSER","sf-
context-current-role":"TESTROLE","sf-context-current-
account":"TESTACCT"},"body":{"\"data\":[[0,\"tok_abc123\"]]}}' \
  --region ${AWS_REGION} \
  response.json

cat response.json
# Expected: {"statusCode":200,"body":{"\"data\":[[0,\"John Doe\"]]}}"}
```

Test in Snowflake

```
-- Test tokenization
SELECT TOK_NAME('John Doe') AS token;
-- Should return: tok_abc123...

-- Test detokenization
SELECT DETOK_NAME(TOK_NAME('John Doe')) AS original;
-- Should return: John Doe

-- Test all data types
SELECT
  TOK_NAME('Jane Smith') as name_token,
  TOK_ID('12345') as id_token,
  TOK_DOB('1990-01-01') as dob_token,
  TOK_SSN('123-45-6789') as ssn_token;

-- Test batch processing
SELECT
  customer_id,
  TOK_NAME(name) as name_token,
  TOK_SSN(ssn) as ssn_token
FROM your_table
LIMIT 100;
```

Check CloudWatch Logs

```
# View Lambda logs
aws logs tail /aws/lambda/skyflow-tokenization \
  --region ${AWS_REGION} \
  --follow

# Look for:
```

```
# - "Loading configuration..."
# - "Loading from AWS Secrets Manager: ..."
# - "Initialized singleton Secrets Manager client" (first invocation only)
# - "Configuration loaded from Secrets Manager"
# - "Credentials type: Service Account (JWT)" or "Credentials type: API Key"
# - "Configuration validated successfully"
# - "Config cache expired, reloading from Secrets Manager" (after 5 minutes)
```

Performance Features

- Both tokenization and detokenization deduplicate input values/tokens within each Lambda batch, after applying all configured transformations.

This implementation includes several optimizations for improved performance and reliability:

AWS Credential Management

- **Singleton Secrets Manager client:** Creates a single AWS Secrets Manager client per Lambda container
- Leverages AWS SDK's built-in credential refresh mechanisms
- Prevents race conditions during AWS IAM credential rotation
- Reduces API calls and improves cold start performance

Configuration Caching with TTL

- Configuration is cached for 5 minutes within each Lambda container
- Allows Skyflow credential rotation without Lambda restart
- Automatic reload after cache expiry
- **Promise-based locking** prevents multiple concurrent config loads during cache expiry
- Balances freshness with performance

Secrets Manager Resilience

- Exponential backoff with jitter for transient AWS errors (prevents thundering herd)
- Automatic retry for credential-related errors (InvalidSignatureException, ExpiredTokenException)
- Smart error classification (retryable vs non-retryable)
- 5xx server errors automatically retried (max 3 attempts)
- These retries ensure AWS infrastructure issues don't disrupt your operations

Skyflow SDK Integration

This implementation uses the **official Skyflow Node.js SDK v2.0.0**, which provides enterprise-grade features managed for you:

- **HTTP/2 connection management:** SDK handles connection multiplexing and header compression automatically
- **Built-in error handling and retry logic:**

- Retries transient errors (network, HTTP 5xx, HTTP 429) with exponential backoff.
- Does **not** retry permanent errors (HTTP 404, 400, 401, 403, etc.).
- All retry logic is managed by the SDK; you do not need to implement custom retries for standard use cases.
- **Optimized request batching:** Efficient payload formatting and parsing
- **Multi-vault support:** Seamlessly routes requests to correct vaults based on data type
- **Lazy vault initialization:** SDK clients are created on-demand per data type, reducing cold start time and memory usage
- **Both authentication methods:** Supports JWT (Service Account) and API Key out of the box

These SDK features work transparently—you get the performance benefits without additional configuration.

Batch Processing

- Separate batch sizes for tokenization and detokenization
- Independent concurrency control for each operation
- Optimized for different workload patterns

 Configuration Options

Data Transformations

The Lambda function supports data transformations that are applied before tokenization. Each vault definition can specify transformation rules:

```
{
  "vaultId": "YOUR_VAULT_ID",
  "table": "persons",
  "column": "name",
  "dataType": "NAME",
  "transformations": {
    "uppercase": true,
    "minLength": 3,
    "stripPunctuation": true
  }
}
```

Transformation Rules by Data Type:

Data Type	Uppercase	Strip Punctuation	Min Length	Validation
NAME	✔ Yes	✔ Yes	3 chars	None
ID	✔ Yes	✔ Yes	3 chars	None
DOB	✗ No	✔ Yes	None	Date range: 0600-01-01 to 3337-11-27

Data Type	Uppercase	Strip Punctuation	Min Length	Validation
SSN	 No	 Yes	3 chars	None

How Transformations Work:

- 1. **Strip Punctuation** - Removes all non-alphanumeric characters (except spaces) before tokenization
 - Example: "O'Brien" → "OBrien"
 - Example: "123-45-6789" → "123456789"
- 2. **Min Length Check** - Values with fewer than the specified alphanumeric characters skip tokenization
 - Only alphanumeric characters count toward length (punctuation is ignored)
 - Short values return the **original input** unchanged (not uppercased)
 - Example: "Jo" (2 chars) → "Jo" (original value returned)
 - Example: "N/A" (2 chars after stripping) → "N/A" (original value returned)
- 3. **Uppercase** - Converts to uppercase before tokenization (NAME and ID only)
 - Ensures consistent tokens: "abc", "Abc", "ABC" all produce the same token
 - Example: "john doe" → "JOHN DOE" → token
 - **Not applied to short values** that skip tokenization
- 4. **DOB Validation** - Validates date is within acceptable range (DOB only)
 - Dates outside range: return original value unchanged
 - Invalid date formats: return original value unchanged

Processing Order:

- 1. Validate date range (DOB only, must happen before stripping punctuation)
- 2. Strip punctuation (if enabled)
- 3. Check minimum length (if specified) - if too short, return original and skip tokenization
- 4. Apply uppercase (if enabled)
- 5. Tokenize processed value

Performance Settings

You can adjust batch sizes and concurrency limits in the Secrets Manager configuration:

```
{
  "credentials": { ... },
  "vaults": { ... },
  "tokenizeBatchSize": 5,
  "tokenizeMaxConcurrency": 400,
  "detokenizeBatchSize": 300,
  "detokenizeMaxConcurrency": 400,
  "logLevel": "INFO"
}
```

Configuration Parameters:

- **tokenizeBatchSize**: Number of records to tokenize per batch (default: 5)
- **tokenizeMaxConcurrency**: Maximum concurrent tokenization batches (default: 400)
- **detokenizeBatchSize**: Number of records to detokenize per batch (default: 300)
- **detokenizeMaxConcurrency**: Maximum concurrent detokenization batches (default: 400)
- **logLevel**: Logging verbosity - **INFO**, **DEBUG**, or **ERROR** (default: INFO)

Tuning Guidelines:

- **Low latency tokenization**: Smaller batch size (5-10), higher concurrency (400-500)
- **High throughput detokenization**: Larger batch size (300-500), higher concurrency (300-400)
- **Rate limit sensitive**: Lower concurrency (100-200) for both operations
- **Balanced workload**: Default values work well for most use cases

Note: Configuration changes are picked up within 5 minutes due to the cache TTL. No Lambda redeployment is needed.

To update configuration:

```
# Update your local skyflow-config.json file, then:
aws secretsmanager update-secret \
  --secret-id ${SECRET_NAME} \
  --secret-string file://skyflow-config.json \
  --region ${AWS_REGION}

# Changes will be applied within 5 minutes (cache TTL)
```

Troubleshooting

Lambda Can't Read Secret

Error: Missing vaultUrl in vaults configuration or Failed to load configuration from Secrets Manager

Solution:

```
# Check Lambda environment variables
aws lambda get-function-configuration \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --query 'Environment.Variables'

# Should show:
# {
#   "SECRETS_MANAGER_SECRET_NAME": "your-secret-name"
# }
```

```
# If missing or incorrect, set it:
aws lambda update-function-configuration \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --environment Variables="{SECRETS_MANAGER_SECRET_NAME=${SECRET_NAME}}"
```

Permission Denied

Error: `AccessDeniedException` when calling `GetSecretValue`

Solution: Check Lambda role has Secrets Manager permissions:

```
# Check the IAM policy
aws iam get-role-policy \
  --role-name skyflow-tokenization-lambda-role \
  --policy-name SecretsManagerReadPolicy

# Verify the Resource ARN matches your secret name pattern
```

HTTP 401 Unauthorized

Error: `HTTP 401: Unauthorized` or authentication failures

Solution: Verify your Skyflow credentials are valid in Secrets Manager:

```
# View secret structure (be careful - shows sensitive data)
aws secretsmanager get-secret-value \
  --secret-id ${SECRET_NAME} \
  --region ${AWS_REGION} \
  --query 'SecretString' \
  --output text | jq '.credentials'

# For JWT authentication, check:
# - clientID and privateKey are present
# - privateKey format is correct (PEM format with \n line breaks)
# - Service account has not been deleted or disabled in Skyflow

# For API Key authentication, check:
# - apiKey is present and valid
# - API key has not been revoked in Skyflow
```

Snowflake Functions Not Working

Error: `Request failed for external function`

Check these:

1. API Gateway URL is correct in function definition
2. Snowflake IAM role trust policy is updated
3. API Gateway deployment was created (**prod** stage)
4. Lambda has permission for API Gateway to invoke it

```
# Check Lambda permissions
aws lambda get-policy \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --query 'Policy' \
  --output text | jq .
```

Invalid Security Token

Error: The security token included in the request is invalid when accessing Secrets Manager

Cause: This error typically occurs during AWS IAM credential rotation. The implementation uses a singleton Secrets Manager client that works with AWS SDK's credential refresh to prevent this issue.

Solution:

1. Check Lambda execution role has correct permissions:

```
aws iam get-role \
  --role-name skyflow-tokenization-lambda-role \
  --query 'Role.Arn'
```

2. Verify the role hasn't been deleted or policies detached
3. If error persists, check CloudWatch logs for more context:

```
aws logs tail /aws/lambda/skyflow-tokenization \
  --region ${AWS_REGION} \
  --follow
```

The singleton client pattern should prevent most occurrences of this error by allowing AWS SDK to manage credential lifecycle.

High Latency

Solution: Adjust performance settings in **skyflow-config.json**:

```
{
  "credentials": { ... },
```

```
"vaults": { ... },
"tokenizeBatchSize": 10,
"tokenizeMaxConcurrency": 500,
"detokenizeBatchSize": 300,
"detokenizeMaxConcurrency": 400
}
```

Then update the secret:

```
aws secretsmanager update-secret \
  --secret-id ${SECRET_NAME} \
  --secret-string file://skyflow-config.json \
  --region ${AWS_REGION}
```

Wait up to 5 minutes for the cache to refresh, then test again.

Updating the Lambda

To update the Lambda code:

```
# Make code changes, then rebuild
npm install --production
zip -r function.zip config.js skyflow-client.js handler.js package.json
node_modules/

# Update Lambda
aws lambda update-function-code \
  --function-name skyflow-tokenization \
  --zip-file fileb://function.zip \
  --region ${AWS_REGION}
```

Cleanup / Uninstall

To remove all resources:

```
# Delete Lambda function
aws lambda delete-function \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION}

# Delete API Gateway
aws apigateway delete-rest-api \
  --rest-api-id ${API_ID} \
  --region ${AWS_REGION}
```



```
# Delete IAM roles
aws iam delete-role-policy \
  --role-name skyflow-tokenization-lambda-role \
  --policy-name SecretsManagerReadPolicy

aws iam detach-role-policy \
  --role-name skyflow-tokenization-lambda-role \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole

aws iam delete-role \
  --role-name skyflow-tokenization-lambda-role

aws iam delete-role-policy \
  --role-name SnowflakeAPIRole \
  --policy-name APIGatewayInvokePolicy

aws iam delete-role \
  --role-name SnowflakeAPIRole

# Delete secret
aws secretsmanager delete-secret \
  --secret-id ${SECRET_NAME} \
  --force-delete-without-recovery \
  --region ${AWS_REGION}

# In Snowflake:
DROP FUNCTION IF EXISTS TOK_NAME(VARCHAR);
DROP FUNCTION IF EXISTS TOK_ID(VARCHAR);
DROP FUNCTION IF EXISTS TOK_DOB(VARCHAR);
DROP FUNCTION IF EXISTS TOK_SSN(VARCHAR);
DROP FUNCTION IF EXISTS DETOK_NAME(VARCHAR);
DROP FUNCTION IF EXISTS DETOK_ID(VARCHAR);
DROP FUNCTION IF EXISTS DETOK_DOB(VARCHAR);
DROP FUNCTION IF EXISTS DETOK_SSN(VARCHAR);
DROP API INTEGRATION IF EXISTS skyflow_api_integration;
```

Support

For issues or questions:

1. Check CloudWatch logs: `aws logs tail /aws/lambda/skyflow-tokenization --region ${AWS_REGION} --follow`
2. Review this troubleshooting section
3. Check Snowflake query history for error messages
4. Contact Skyflow support with CloudWatch log excerpts

Appendix: Manual Environment Variables (Method 2)

This section documents **Method 2: Lambda Environment Variables** as an alternative to AWS Secrets Manager. All features (transformations, caching, etc.) work identically with both methods.

When to Use This Method

- Development/testing environments
- Simpler deployments without Secrets Manager
- Avoiding additional AWS service costs
- Full control via AWS CLI

Environment Variable Reference

The Lambda function supports the following environment variables:

Authentication (choose one):

Option A: JWT (Service Account) - Recommended

```
SKYFLOW_CLIENT_ID="your_client_id"
SKYFLOW_CLIENT_NAME="your_service_account_name"
SKYFLOW_TOKEN_URI="https://manage.skyflowapis.com/v1/auth/sa/oauth/token"
SKYFLOW_KEY_ID="your_key_id"
SKYFLOW_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nYOUR_KEY_HERE\n-----END
PRIVATE KEY-----\n"
SKYFLOW_KEY_ALGORITHM="KEY_ALG_RSA_2048"
```

Option B: API Key

```
SKYFLOW_API_KEY="your_api_key"
```

Vault Configuration (required):

```
SKYFLOW_VAULT_URL="https://YOUR_CLUSTER_ID.vault.skyflowapis.com"
SKYFLOW_VAULT_DEFINITIONS=' [{"vaultId": "...", "table": "...", "column": "...",
"dataType": "NAME", "transformations":
{"uppercase": true, "minLength": 3, "stripPunctuation": true}}, ... ] '
```

Performance Settings (required):

```
SKYFLOW_TOKENIZE_BATCH_SIZE="5"
SKYFLOW_TOKENIZE_MAX_CONCURRENCY="400"
SKYFLOW_DETOKENIZE_BATCH_SIZE="300"
SKYFLOW_DETOKENIZE_MAX_CONCURRENCY="400"
SKYFLOW_LOG_LEVEL="INFO"
```

Step-by-Step: Deploy with Environment Variables

Step 1: Prepare your configuration

Create a file `env-vars.json` with your settings:

```
{
  "SKYFLOW_CLIENT_ID": "your_client_id",
  "SKYFLOW_CLIENT_NAME": "your_service_account_name",
  "SKYFLOW_TOKEN_URI":
  "https://manage.skyflowapis.com/v1/auth/sa/oauth/token",
  "SKYFLOW_KEY_ID": "your_key_id",
  "SKYFLOW_PRIVATE_KEY": "-----BEGIN PRIVATE KEY-----\\nYOUR_KEY_HERE\\n--
  ---END PRIVATE KEY-----\\n",
  "SKYFLOW_KEY_ALGORITHM": "KEY_ALG_RSA_2048",
  "SKYFLOW_VAULT_URL": "https://YOUR_CLUSTER_ID.vault.skyflowapis.com",
  "SKYFLOW_VAULT_DEFINITIONS": "
  [{\"vaultId\": \"vault1\", \"table\": \"persons\", \"column\": \"name\", \"dataT
  ype\": \"NAME\", \"transformations\":
  {\"uppercase\": true, \"minLength\": 3, \"stripPunctuation\": true}},
  {\"vaultId\": \"vault2\", \"table\": \"persons\", \"column\": \"person_id\", \"d
  ataType\": \"ID\", \"transformations\":
  {\"uppercase\": true, \"minLength\": 3, \"stripPunctuation\": true}},
  {\"vaultId\": \"vault3\", \"table\": \"persons\", \"column\": \"date_of_birth\"
  , \"dataType\": \"DOB\", \"transformations\":
  {\"uppercase\": false, \"stripPunctuation\": true, \"validation\":
  {\"minDate\": \"0600-01-01\", \"maxDate\": \"3337-11-27\"}}},
  {\"vaultId\": \"vault4\", \"table\": \"persons\", \"column\": \"ssn\", \"dataTyp
  e\": \"SSN\", \"transformations\":
  {\"uppercase\": false, \"minLength\": 3, \"stripPunctuation\": true}}}],
  "SKYFLOW_TOKENIZE_BATCH_SIZE": "5",
  "SKYFLOW_TOKENIZE_MAX_CONCURRENCY": "400",
  "SKYFLOW_DETOKENIZE_BATCH_SIZE": "100",
  "SKYFLOW_DETOKENIZE_MAX_CONCURRENCY": "500",
  "SKYFLOW_LOG_LEVEL": "ERROR"
}
```

Important: The `SKYFLOW_VAULT_DEFINITIONS` must be a JSON string (escaped quotes). Each vault definition should include the `transformations` object.

Step 2: Follow main deployment guide with modifications

1. **Skip Step 3** (no Secrets Manager needed)
2. **Skip Step 4.3** (no Secrets Manager permissions needed)
3. **Modify Step 4.2** - Create Lambda role WITHOUT Secrets Manager policy:

```
# Create IAM role
aws iam create-role \
  --role-name skyflow-tokenization-lambda-role \
```

```

--assume-role-policy-document file://lambda-trust-policy.json \
--description "Execution role for Skyflow tokenization Lambda"

# Attach basic Lambda execution policy
aws iam attach-role-policy \
  --role-name skyflow-tokenization-lambda-role \
  --policy-arn arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole

# Wait for role to propagate
sleep 10

```

4. **Continue with Step 5** (build deployment package as documented)

5. **Modify Step 6** - Create Lambda with environment variables instead of secret:

```

# Create Lambda function with environment variables
aws lambda create-function \
  --function-name skyflow-tokenization \
  --runtime nodejs20.x \
  --role arn:aws:iam::${AWS_ACCOUNT_ID}:role/skyflow-tokenization-
lambda-role \
  --handler handler.handler \
  --zip-file fileb://function.zip \
  --timeout 60 \
  --memory-size 256 \
  --description "Skyflow tokenization and detokenization for Snowflake"
\
  --environment file://env-vars.json \
  --region ${AWS_REGION}

# Verify Lambda was created
aws lambda get-function-configuration \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --query 'Environment.Variables'

```

6. **Continue with Steps 7-9** (API Gateway and Snowflake setup as documented)

Updating Configuration

To update configuration with this method:

```

# Update environment variables
aws lambda update-function-configuration \
  --function-name skyflow-tokenization \
  --environment file://env-vars.json \
  --region ${AWS_REGION}

```

```
# Wait for update to complete
aws lambda wait function-updated \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION}
```

Note: Configuration changes require Lambda redeployment, unlike Secrets Manager which updates within 5 minutes via cache refresh.

Validation

After deployment, verify environment variables are set:

```
aws lambda get-function-configuration \
  --function-name skyflow-tokenization \
  --region ${AWS_REGION} \
  --query 'Environment.Variables' \
  --output json
```

Check CloudWatch logs to confirm it's using environment variables:

```
aws logs tail /aws/lambda/skyflow-tokenization \
  --region ${AWS_REGION} \
  --follow

# Look for: "Loading from SKYFLOW_* environment variables"
```

Troubleshooting

Error: "Missing Skyflow credentials"

- Ensure either `SKYFLOW_CLIENT_ID` or `SKYFLOW_API_KEY` is set
- Check for typos in environment variable names (must be uppercase)

Error: "Missing vaultUrl in vaults configuration"

- Verify `SKYFLOW_VAULT_URL` is set correctly
- Ensure `SKYFLOW_VAULT_DEFINITIONS` is valid JSON (use a JSON validator)

Error: "Invalid vaultUrl format"

- Vault URL must match pattern: `https://<clusterId>.vault.skyflowapis.com`

Feature Compatibility

All features work identically with both configuration methods:

Feature	Secrets Manager	Environment Variables
---------	-----------------	-----------------------

Feature	Secrets Manager	Environment Variables
Data transformations	✓	✓
DOB validation	✓	✓
JWT/API Key auth	✓	✓
Batch processing	✓	✓
Config updates	Auto (5-min)	Manual (redeploy)

 Additional Resources

- **Skyflow API Documentation:** <https://docs.skyflow.com/>
- **AWS Lambda Documentation:** <https://docs.aws.amazon.com/lambda/>
- **AWS Secrets Manager:** <https://docs.aws.amazon.com/secretsmanager/>
- **Snowflake External Functions:** <https://docs.snowflake.com/en/sql-reference/external-functions-introduction>

Deployment complete! 🎉

Your Skyflow tokenization is now integrated with Snowflake using a Node.js Lambda function with performance optimizations including configuration caching, data transformations, and adaptive retry logic.